EC991 Topics in Macroeconomics

Week 2 Lab · Dynamic Programming and Value Function Iteration

Aseem Patel

Department of Economics, University of Essex

Today's lab

1. The basics of value function iteration

- From the sequence problem to the Bellman equation; the state; the Euler equation; the algorithm

2. An infinite-horizon problem

- Iterate the Bellman equation to its fixed point, read off the policy functions, test them against theory

3. A finite-horizon problem

- Backward induction from the last period; validate against the cake-eating closed form

4. Two interactive pages

- **VFI on a grid**: every Bellman step written out on a small grid, then the policy functions
- **Backward induction on a grid**: the same table, one period at a time, from T back to 1

5. The code

- Pull the package `EC991Week2`; the code guide explains every folder and file

From the lecture to the computer

The lecture's household problem, and what we know about its solution:

$$\max_{\{a_{t+1}\}} \mathbb{E}_0 \sum_{t=0}^{\infty} \beta^t u(c_t) \quad \text{s.t.} \quad c_t + a_{t+1} \leq w y_t + R a_t, \qquad a_{t+1} \geq -\phi$$

- The Euler equation $u'(c_t) = \beta R \mathbb{E}_t[u'(c_{t+1})] + \mu_t$ **characterises** the optimum
- It does not tell us **what the household does**: how much it saves at each level of wealth
- For that we need to **compute** the solution. That is today's job

One simplification for today.

- No income risk: $y_t = y$ constant, wage normalised to one (risk returns in week 3)

What does “recursive” mean?

- **Recursive**: express today’s problem in terms of the same problem tomorrow
- The household chooses consumption and saving today; its saving becomes tomorrow’s assets
- Tomorrow it faces the same decision again, starting from those assets

Three steps.

1. **Sequence formulation**: given a_0, choose the entire plan $a_1, a_2, a_3, \ldots$ to maximise lifetime utility
2. **Summarise the future with a function**: $V(a)$ is the highest lifetime utility attainable from assets a
   - it already incorporates every optimal decision the household will make from that point on
3. **Choose today’s saving, accounting for its future value**: save a', consume $y + Ra - a'$, start tomorrow with a'

- Tomorrow’s problem has the same structure as today’s; only the starting assets change
  - that is why the **same function** V appears on both sides

From the sequence problem to the recursive formulation

Let $V(a_0)$ be the best lifetime utility attainable from assets a_0.

Since u is increasing, the budget binds: $c_t = y + Ra_t - a_{t+1}$. For $c_t > 0$, define

$$\Gamma(a) = \{b : b \geq -\phi,\ y + Ra - b > 0\} \qquad \text{(feasible saving from assets } a\text{)}.$$

Split lifetime utility into today and everything after:

$$\begin{aligned}
V(a_0) &= \max_{\substack{\{a_{t+1}\}_{t\geq 0} \\ a_{t+1}\in\Gamma(a_t),\ t\geq 0}} \Big[u(c_0) + \beta \sum_{t=1}^{\infty} \beta^{t-1} u(c_t)\Big] \\
&= \max_{a_1\in\Gamma(a_0)} \Big[u(y + Ra_0 - a_1) + \beta \max_{\substack{\{a_{t+1}\}_{t\geq 1} \\ a_{t+1}\in\Gamma(a_t),\ t\geq 1}} \sum_{t=1}^{\infty} \beta^{t-1} u(c_t)\Big] \\
&= \max_{a_1\in\Gamma(a_0)} \Big[u(y + Ra_0 - a_1) + \beta V(a_1)\Big].
\end{aligned}$$

- Line 2: **given** a_1, optimise future choices subject to all future constraints
- Line 3: that continuation problem starts from a_1, so its value is $V(a_1)$
- Write $a_0 = a$ and $a_1 = a'$: this is the **Bellman equation**

The argument of V is the state variable

Definition (informal). The **state** is the minimal information needed today to

1. evaluate current payoffs and constraints, and
2. forecast everything relevant about the future

Today the state is a alone:

- a enters the budget constraint, and income is the same known y at every date
- History does not matter: two households with the same a face identical problems from here on

In the lecture's model the state is the pair (a, y):

- y enters the budget constraint and, through P, forecasts future income
- Identifying the state correctly is half the work in every model of this course

Value function and policy function

For each current asset level a, compare all feasible saving choices $a' \in \Gamma(a)$.

Value function: how well can the household do?

$$V(a) = \max_{a' \in \Gamma(a)} \Big\{ u(y + Ra - a') + \beta V(a') \Big\}$$

- $V(a)$ is the **maximum value**: the best lifetime utility attainable from assets a

Policy function: what should the household choose?

$$g(a) = \arg\max_{a' \in \Gamma(a)} \Big\{ u(y + Ra - a') + \beta V(a') \Big\}$$

- $g(a)$ is the **maximising choice**: the saving a' that achieves that value
- Repeat for every a: g is a saving rule, with consumption $c(a) = y + Ra - g(a)$

On the asset grid

- The highest objective value in each row gives $V(a)$
- The saving choice associated with that entry gives $g(a)$

Time subscripts on the state variable

Sometimes people write

$$V(a_t) = \max_{c_t,\, a_{t+1}} u(c_t) + \beta V(a_{t+1}) \quad \text{s.t.} \quad c_t + a_{t+1} \leq y + Ra_t, \qquad a_{t+1} \geq -\phi$$

Please don't do this!

- V is a function of a **level of assets**, whatever the date at which it is held
- The subscript suggests a different function at each date; in the stationary problem there is **one**
- When V really does change with the date, the subscript goes on V, not on a (finite horizon, below)

Euler equation from the Bellman equation

Form the Lagrangean, with λ on the budget and μ on the borrowing constraint:

$$\mathcal{L} \;=\; u(c) + \beta V(a') + \lambda\,[\,y + Ra - c - a'\,] + \mu\,[\,a' + \phi\,]$$

First-order conditions and the envelope condition:

$$c: \;\; u'(c) = \lambda, \qquad a': \;\; \beta V'(a') = \lambda - \mu, \qquad \text{envelope: } V'(a) = \lambda R \;\Rightarrow\; V'(a') = \lambda' R$$

Substitute the envelope condition into the FOC for a', then use $u'(c) = \lambda$:

$$u'(c) \;=\; \beta R\, u'(c') + \mu \qquad \Longrightarrow \qquad u'(c) \;\geq\; \beta R\, u'(c')$$

- Equality when the borrowing constraint is slack ($\mu = 0$); the lecture's Euler equation without the expectation
- The inequality is a **testable prediction** for your code: check it along a simulated path

Value function iteration: the idea

- The **easiest** method to solve the Bellman equation numerically for $V(a)$
- **Guess** a value function on the right-hand side, **maximise** to get the left-hand side
- **Update** the guess with the result and repeat until the function stops changing
- Converges whenever $\beta < 1$, from any starting guess: the Bellman operator is a **contraction mapping** (week 3)
- A computer cannot hold a function: hold its **values on a grid** of asset levels instead
- Simplest method, and the **slowest**: week 3 is about doing better

Value function iteration: the algorithm

Step 1. Discretise the asset space $\mathcal{A} = \{a_1, \ldots, a_{n_a}\}$; set $a_1 = -\phi$

Step 2. Guess the initial $V_0(a)$. A good guess: the value of consuming $ra + y$ forever,

$$V_0(a) \;=\; \sum_{t=0}^{\infty} \beta^t u(ra + y) \;=\; \frac{u(ra + y)}{1 - \beta}$$

Step 3. Set $\ell = 1$. For every $a_i \in \mathcal{A}$ solve, with a' restricted to grid points,

$$a'_\ell(a_i) = \arg\max_{a' \in \mathcal{A}} \; u\big(y + Ra_i - a'\big) + \beta V_{\ell-1}(a'), \qquad V_\ell(a_i) = \text{the maximised value}$$

Step 4. If $\max_i |V_\ell(a_i) - V_{\ell-1}(a_i)| <$ tolerance: stop. Else $\ell \to \ell + 1$, back to Step 3

- Step 3 applies the Bellman right-hand side once; Step 4 asks whether we have reached the fixed point
- The converged V and $a'(a)$ are the solution; $c(a) = y + Ra - a'(a)$ follows

Step 3 is a double loop

For each state a_i, try every candidate a'_k on the grid and keep the best one:

```
for i in 1:na                          # state a_i
    best, kbest = -Inf, 0
    for k in 1:na                      # candidate a'_k
        c = R * a[i] + y - a[k]
        c <= 0 && continue             # infeasible: skip it
        v = u(c) + beta * V0[k]        # the Bellman right-hand side
        v > best && (best, kbest = v, k)
    end
    V1[i], pol[i] = best, kbest
end
```

- The Bellman equation is not an abstract object: it is the line `v = u(c) + beta * V0[k]`
- Infeasible consumption gets value $-\infty$, so it can never be chosen
- `pol[i]` stores the **index** of the chosen a'; the policy is `a[pol]`
- Wrap this in the Step 4 convergence loop and the solver is complete

From the converged V to the policy functions

VFI returns numbers $V^*(a_i)$ on the grid. The policies come from **one more maximisation** with V^* on the right:

$$k^*(i) \;=\; \arg\max_{k} \; u\big(y + Ra_i - a_k\big) + \beta\, V^*(a_k), \qquad a'(a_i) = a_{k^*(i)}, \qquad c(a_i) = y + Ra_i - a'(a_i)$$

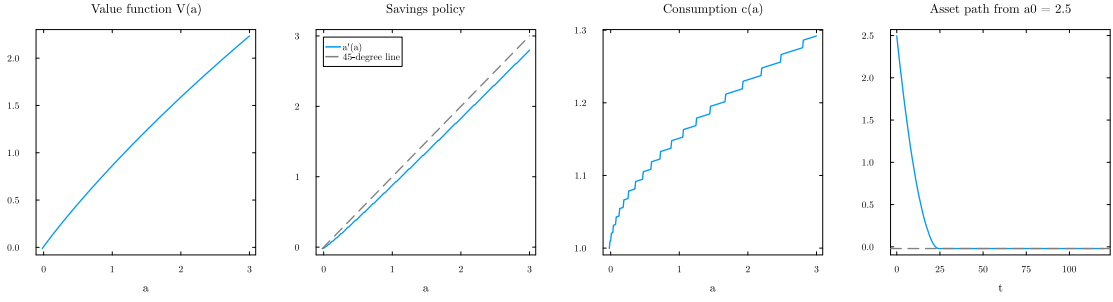
- No extra work in practice: the last pass of the double loop already stored $k^*(i)$ in `pol`
- `pol` holds **indices**; turn them into asset levels, then use the budget for consumption:

```
apol = a[pol]                  # a'(a_i), always a grid point
cpol = y .+ R .* a .- apol     # c(a_i), from the budget constraint
```

Check the policies before using them.

- $c(a_i) > 0$ everywhere, and $a'(a)$ increasing in a
- With $\beta R < 1$: $a'(a) < a$ above the borrowing limit, so the household dissaves
- $a'(a_{n_a}) < a_{n_a}$: otherwise the top of the grid, not the household, chose the savings
- To simulate, follow the indices: $i_{t+1} = \texttt{pol}[i_t]$, so the path never leaves the grid

What the solution looks like, and how to test it



$\beta = 1/1.05$, $R = 1.03$, $y = 1$, $\phi = 0.02$, $\gamma = 2$; **300 grid points on** $[-\phi, 3]$; **produced by EC991Week2/run/make_figures.jl**

Two predictions from theory to test the output against:

- With $\beta R < 1$ the asset path **drifts down to the borrowing limit** and stays there
- Along the path $u'(c_t) \geq \beta R\, u'(c_{t+1})$, with equality until the limit binds
- The policy $a'(a)$ is restricted to grid points, so $c(a)$ is a step function: **not very accurate**
- That inaccuracy is the problem week 3 solves

Finite horizon: now V carries a time subscript

The household lives T periods and income y_t is a known sequence. The horizon shortens as time passes, so

$$V_t(a) \;=\; \max_{a' \geq -\phi} \; u\big(y_t + Ra - a'\big) + \beta V_{t+1}(a'), \qquad V_{T+1}(a) \equiv 0$$

- Solution: a **sequence** of value functions $\{V_t\}_{t=1}^{T}$ and policies $\{a'_t(a), c_t(a)\}_{t=1}^{T}$
  - the same assets mean different things at different ages: a household at $t = 1$ has T periods to use them
  - so the policy depends on age too: the saving rule is $a'_t(a)$, a **life-cycle** profile
- Terminal condition: no value beyond death (stretch: a bequest motive, $V_{T+1}(a) = \phi_b\, u(a)$)
  - in the last period the household consumes everything it can: $a'_T(a) = -\phi$ and $V_T(a) = u(y_T + Ra + \phi)$
- The subscript is on V, not on a: the state is still a level of assets

The problem solves itself.

- At $t = T$ the continuation is known (zero), so V_T is a one-dimensional max: **no fixed point**
- Then V_T is known, so V_{T-1} follows the same way; walk back to $t = 1$: **backward induction**
- The right-hand side is the **same operator** as before, applied to V_{t+1} instead of to V itself
  - with constant income, V_t depends only on the periods left, $T - t + 1$
  - backward induction from zero is VFI from the zero guess: early V_t of a long life approach V

Backward induction on a grid

Same asset grid, a' again restricted to grid points. Store V as an $n_a \times (T+1)$ array.

Step 1. Set $V_{T+1}(a_i) = 0$ for all i

Step 2. For $t = T, \ldots, 1$ and each i: for every candidate k, compute $c_{ik} = Ra_i + y_t - a_k$;
value $= u(c_{ik}) + \beta V_{t+1}(a_k)$ if $c_{ik} > 0$, else $-\infty$; take the max, record $k^*(i,t)$

Step 3. Simulate forward from a_1 using the recorded policy

Compared with the infinite-horizon code.

- The inner double loop is **identical**; the outer loop runs over t instead of over iterations ℓ
- No convergence check: after T passes you are done, by construction
- The continuation value V_{t+1} is exact, not a guess; nothing is being iterated to a fixed point

Validate against a known answer: cake eating

Never trust numerical output you have not tested. A special case with a closed form: $y_t = 0$, $R = 1$, $u = \log$.

- Euler: $u'(c_t) = \beta\, u'(c_{t+1}) \;\Rightarrow\; c_{t+1} = \beta c_t$
- Remaining budget: $\sum_{s=t}^{T} c_s = a_t \;\Rightarrow\; c_t \sum_{s=0}^{T-t} \beta^s = a_t$

$$c_t \;=\; \frac{1-\beta}{1-\beta^{\,T-t+1}}\; a_t$$

- Two lines of algebra: a complete test of several dozen lines of code
- Your code will track it closely, **not exactly**: the gap is **grid coarseness**
- Check the gap shrinks as n_a grows; if it does not, the bug is in the code, not in the grid